

Delegate - C++14

Generated by Doxygen 1.16.1

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 bricxx Namespace Reference	9
6 Class Documentation	11
6.1 bricxx::BadDelegateCall Class Reference	11
6.1.1 Detailed Description	11
6.1.2 Member Function Documentation	11
6.1.2.1 what()	11
6.2 bricxx::Delegate< TSignature > Class Template Reference	12
6.2.1 Detailed Description	12
6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference	13
6.3.1 Detailed Description	15
6.3.2 Constructor & Destructor Documentation	15
6.3.2.1 Delegate() [1/2]	15
6.3.2.2 Delegate() [2/2]	15
6.3.2.3 ~Delegate()	15
6.3.3 Member Function Documentation	16
6.3.3.1 operator=()	16
6.3.3.2 operator==(())	16
6.3.3.3 operator!=(())	16
6.3.3.4 Bind() [1/3]	16
6.3.3.5 Bind() [2/3]	16
6.3.3.6 Bind() [3/3]	16
6.3.3.7 IsBindedTo() [1/3]	17
6.3.3.8 IsBindedTo() [2/3]	17
6.3.3.9 IsBindedTo() [3/3]	17
6.3.3.10 operator()()	17
6.3.3.11 CreateAndBind() [1/3]	18
6.3.3.12 CreateAndBind() [2/3]	18
6.3.3.13 CreateAndBind() [3/3]	18
6.4 bricxx::Delegate< TRetVal(TParams...) > Class Template Reference	18
6.4.1 Detailed Description	20

6.4.2 Constructor & Destructor Documentation	20
6.4.2.1 Delegate() [1/2]	20
6.4.2.2 Delegate() [2/2]	20
6.4.2.3 ~Delegate()	20
6.4.3 Member Function Documentation	20
6.4.3.1 operator=()	20
6.4.3.2 operator==()	20
6.4.3.3 operator!=(())	21
6.4.3.4 Bind() [1/3]	21
6.4.3.5 Bind() [2/3]	21
6.4.3.6 Bind() [3/3]	21
6.4.3.7 IsBindedTo() [1/3]	21
6.4.3.8 IsBindedTo() [2/3]	21
6.4.3.9 IsBindedTo() [3/3]	22
6.4.3.10 operator()()	22
6.4.3.11 CreateAndBind() [1/3]	22
6.4.3.12 CreateAndBind() [2/3]	22
6.4.3.13 CreateAndBind() [3/3]	23
6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference	23
6.5.1 Detailed Description	24
6.5.2 Member Typedef Documentation	24
6.5.2.1 ErasedObjectType	24
6.5.2.2 StubFnPtr	24
6.5.3 Constructor & Destructor Documentation	24
6.5.3.1 DelegateBase() [1/2]	24
6.5.3.2 DelegateBase() [2/2]	25
6.5.3.3 ~DelegateBase()	25
6.5.4 Member Function Documentation	25
6.5.4.1 IsBinded()	25
6.5.4.2 operator bool()	25
6.5.4.3 Unbind()	25
6.5.4.4 Reset()	25
6.5.4.5 operator=()	25
6.5.4.6 operator==()	26
6.5.4.7 operator!=(())	26
6.5.4.8 NullStub()	26
6.5.5 Member Data Documentation	26
6.5.5.1 pStub_	26
6.5.5.2 pTypeErasedObject_	26
7 File Documentation	27
7.1 delegate.hpp File Reference	27

7.1.1 Macro Definition Documentation	27
7.1.1.1 BRICXX_DELEGATE_API	27
7.2 delegate.hpp	28
Index	37

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

bricxx	9
----------------------------------	---

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

<code>bricxx::Delegate< TSignature ></code>	12
<code>bricxx::DelegateBase< TRetVal, TParams ></code>	23
<code>bricxx::DelegateBase< TRetVal, TParams... ></code>	23
<code>bricxx::Delegate< TRetVal(TParams...) const ></code>	13
<code>bricxx::Delegate< TRetVal(TParams...)></code>	18
<code>std::exception</code>	
<code>bricxx::BadDelegateCall</code>	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

bricxx::BadDelegateCall	
Exception thrown by <code>Delegate::operator()()</code> if the delegate is not binded	11
bricxx::Delegate< TSignature >	
A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined	12
bricxx::Delegate< TRetVal(TParams...) const >	
Delegate partial specialization for const	13
bricxx::Delegate< TRetVal(TParams...) >	
Delegate partial specialization for non-const	18
bricxx::DelegateBase< TRetVal, TParams >	
Base class for Delegate partial specializations	23

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

delegate.hpp		27
------------------------------	-------	----

Chapter 5

Namespace Documentation

5.1 bricxx Namespace Reference

Classes

- class [Delegate](#)
A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.
- class [BadDelegateCall](#)
Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class [DelegateBase](#)
Base class for `Delegate` partial specializations.
- class [Delegate< TRetVal\(TParams...\)>](#)
`Delegate` partial specialization for non-const.
- class [Delegate< TRetVal\(TParams...\) const >](#)
`Delegate` partial specialization for const.

Chapter 6

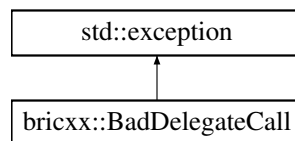
Class Documentation

6.1 bricxx::BadDelegateCall Class Reference

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

```
#include <delegate.hpp>
```

Inheritance diagram for `bricxx::BadDelegateCall`:



Public Member Functions

- `const char * what () const noexcept override`

6.1.1 Detailed Description

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

Definition at line [161](#) of file [delegate.hpp](#).

6.1.2 Member Function Documentation

6.1.2.1 what()

```
const char * bricxx::BadDelegateCall::what () const [override], [noexcept]
```

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

6.2 bricxx::Delegate< TSignature > Class Template Reference

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

6.2.1 Detailed Description

```
template<typename TSignature>
class bricxx::Delegate< TSignature >
```

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

[Delegate](#):

- is copyable, small (two pointers) and fast (no dynamic memory allocation, no virtual functions, all functions are inline (except stubs because their address is needed), delegate call involves only two extra calls (operator>() and stub function)).
- supports (member) functions and functors/lambda.
- is a single header library so it is easy to integrate in any project.

Attention

Because [Delegate](#) is non-owning special care is needed to ensure that the binded target outlive it.

Remarks

- Two delegates are considered equal if they are not binded or binded to the same target.
- Equality comparison and `Delegate::IsBindedTo()` allows delegates to be easily used as call-backs that can be registered/unregistered (`::std::function_ref` is not equality comparable).
- The solutions presented in **references** directory are not const-correct (they have the same problem as `::std::function`).

See also

- [Rich Hickey: Callbacks in C++ Using Template Functors](#)
- Bert Peers: [Callbacks in C++](#)
- Don Clugston, CodeProject: [Member Function Pointers and the Fastest Possible C++ Delegates](#)
- Sergey Ryazanov, CodeProject: [The Impossibly Fast C++ Delegates](#)
- Sergey Alexandrovich Kryukov, CodeProject: [The Impossibly Fast C++ Delegates, Fixed](#)
- Aleksei Trunov, CodeProject: [Yet Another Generalized Functors Implementation in C++](#)
- JaeWook Choi, CodeProject: [Fast C++ Delegate](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 1\)](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 2\)](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 3\)](#)
- [Matthew Rodusek: Unbelievably Fast Delegate](#)

- [user1095108](#): Impossibly fast delegate in C++11
- [user1095108](#): `delegate.hpp`
- `references` directory in this project that contains the above.

Minimal introduction/examples (please see tests for more details):

```
// (static member) function
Delegate<int (int)> functionDelegate;
functionDelegate.Bind<&::islower>();
if (!functionDelegate.IsBindedTo<&::islower>()) throw runtime_error("function test failed");
if (!functionDelegate('a')) throw runtime_error("function test failed");

// lambda/functor
unsigned int numCalls = 0U;
auto lambda = [&numCalls](int x, int y) mutable -> bool {
    ++numCalls;
    return x > y;
};
Delegate<bool (int, int)> lambdaDelegate;
lambdaDelegate.Bind(lambda);
if (!lambdaDelegate.IsBindedTo(lambda)) throw runtime_error("lambda test failed");
if (!lambdaDelegate(2, 1)) throw runtime_error("lambda test failed");
if (numCalls != 1U) throw runtime_error("lambda test failed");

// non-static member function
string hello = "Hello, World!";
Delegate<void ()> methodDelegate;
methodDelegate.Bind<string, &string::clear>(hello);
methodDelegate();
if (!hello.empty()) throw runtime_error("method test failed");

// comparison
Delegate<int (int)> functionDelegate1;
functionDelegate1.Bind<&::isupper>();
if (functionDelegate == functionDelegate1) throw runtime_error("comparison test failed");
functionDelegate1.Bind<&::islower>();
if (functionDelegate != functionDelegate1) throw runtime_error("comparison test failed");

// const-correctness
Delegate<void () const> constDelegate;
//constDelegate.Bind<string, &string::clear>(hello); // OK, does not compile
(void) constDelegate;

// unbind
functionDelegate.Unbind();
if (functionDelegate.IsBinded()) throw runtime_error("unbind test failed");

// BadDelegateCall
bool caught = false;
try {
    functionDelegate('a');
}
catch (const BadDelegateCall&) {
    caught = true;
}
if (!caught) throw runtime_error("BadDelegateCall test failed");
```

Definition at line 154 of file [delegate.hpp](#).

The documentation for this class was generated from the following file:

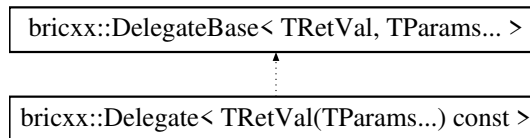
- [delegate.hpp](#)

6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference

[Delegate](#) partial specialization for `const`.

```
#include <delegate.hpp>
```

Inheritance diagram for `bricxx::Delegate< TRetVal(TParams...) const >`:



Public Member Functions

- `~Delegate () noexcept=default`
- `constexpr Delegate & operator= (const Delegate &other) noexcept`
- `template<typename... UParams>
constexpr TRetVal operator() (UParams &&... params) const`

Constructors.

- `constexpr Delegate () noexcept`
Creates a delegate that is not binded.
- `constexpr Delegate (const Delegate &other) noexcept`

Equality operators.

- `constexpr bool operator== (const Delegate &other) const noexcept`
- `constexpr bool operator!= (const Delegate &other) const noexcept`

Binders.

Binds the delegate to a target.

- `template<TRetVal(*) (TParams...) TFunction>
constexpr void Bind () noexcept`
Binds the delegate to a (static member) function.
- `template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
constexpr void Bind (const TFunction &function) noexcept`
Binds the delegate to a functor / lambda (TFunction is auto-deduced).
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
constexpr void Bind (const TClass &object) noexcept`
Binds the delegate to an object + non-static member function.

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- `template<TRetVal(*) (TParams...) TFunction>
constexpr bool IsBindedTo () const noexcept`
Test if the delegate is binded to the given (static member) function.
- `template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
constexpr bool IsBindedTo (const TFunction &function) const noexcept`
Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
constexpr bool IsBindedTo (const TClass &object) const noexcept`
Test if the delegate is binded to the given object + non-static member function.

Static Public Member Functions

Create and bind (convenience methods).

Creates a delegate and binds it to the given target.

- `template<TRetVal(*)(TParams...) TFunction>`
`static constexpr Delegate CreateAndBind () noexcept`
Creates a delegate and binds it to the given (static member) function.
- `template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>`
`static constexpr Delegate CreateAndBind (const TFunction &function) noexcept`
*Creates a delegate and binds it to the given *functor / lambda* (*TFunction* is auto-deduced).*
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>`
`static constexpr Delegate CreateAndBind (const TClass &object) noexcept`
*Creates a delegate and binds it to the given *object* + non-static member function.*

6.3.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) const >
```

[Delegate](#) partial specialization for const.

Definition at line 341 of file [delegate.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.3.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate (
    const Delegate< TRetVal(TParams...) const > & other) [constexpr], [noexcept]
```

6.3.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::~~Delegate () [default], [noexcept]
```

Remarks

Not virtual.

6.3.3 Member Function Documentation

6.3.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...) const >::operator= (
    const Delegate< TRetVal(TParams...) const > & other) [constexpr], [noexcept]
```

6.3.3.2 operator==(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator==( (
    const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.3 operator!=(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator!=( (
    const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.4 Bind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind () [constexpr], [noexcept]
```

Binds the delegate to a (static member) function.

6.3.3.5 Bind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFuncor, typename = typename ::std::enable_if< ::std::is_class<TFuncor>::value >::type>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
    const TFuncor & functor) [constexpr], [noexcept]
```

Binds the delegate to a functor / lambda (TFuncor is auto-deduced).

6.3.3.6 Bind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
    const TClass & object) [constexpr], [noexcept]
```

Binds the delegate to an object + non-static member function.

6.3.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo () const [constexpr], [noexcept]
```

Test if the delegate is binded to the given (static member) function.

6.3.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
    const TFunction & function) const [constexpr], [noexcept]
```

Test if the delegate is binded to the given *function* / *lambda* (TFunction is auto-deduced).

6.3.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
    const TClass & object) const [constexpr], [noexcept]
```

Test if the delegate is binded to the given *object* + non-static member function.

6.3.3.10 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate< TRetVal(TParams...) const >::operator() (
    UParams &&... params) const [constexpr]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (UParams is a deduced type, TParams is not).

Exceptions

<i>BadDelegateCall</i>	if the delegate is not binded.
------------------------	--------------------------------

6.3.3.11 CreateAndBind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
constexpr Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind () [static],
[constexpr], [noexcept]
```

Creates a delegate and binds it to the given (static member) function.

6.3.3.12 CreateAndBind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
constexpr Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind (
    const TFunction & functor) [static], [constexpr], [noexcept]
```

Creates a delegate and binds it to the given functor / lambda (TFunction is auto-deduced).

6.3.3.13 CreateAndBind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
constexpr Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind (
    const TClass & object) [static], [constexpr], [noexcept]
```

Creates a delegate and binds it to the given object + non-static member function.

The documentation for this class was generated from the following file:

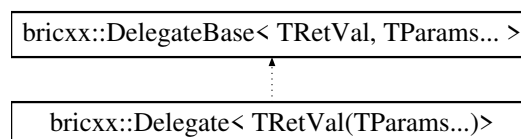
- [delegate.hpp](#)

6.4 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference

[Delegate](#) partial specialization for non-const.

```
#include <delegate.hpp>
```

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...)>:



Public Member Functions

- [~Delegate](#) () noexcept=default
- constexpr [Delegate](#) & [operator=](#) (const [Delegate](#) &other) noexcept
- template<typename... UParams>
constexpr TRetVal [operator\(\)](#) (UParams &&... params)

Constructors.

- constexpr [Delegate](#) () noexcept
Creates a delegate that is not binded.
- constexpr [Delegate](#) (const [Delegate](#) &other) noexcept

Equality operators.

- constexpr bool [operator==](#) (const [Delegate](#) &other) const noexcept
- constexpr bool [operator!=](#) (const [Delegate](#) &other) const noexcept

Binders.

Binds the delegate to a target.

- template<TRetVal(*) (TParams...) TFunction>
constexpr void [Bind](#) () noexcept
Binds the delegate to a (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
constexpr void [Bind](#) (TFunction &function) noexcept
Binds the delegate to a functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
constexpr void [Bind](#) (TClass &object) noexcept
Binds the delegate to an object + non-static member function.

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<TRetVal(*) (TParams...) TFunction>
constexpr bool [IsBindedTo](#) () const noexcept
Test if the delegate is binded to the given (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
constexpr bool [IsBindedTo](#) (TFunction &function) const noexcept
Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
constexpr bool [IsBindedTo](#) (TClass &object) const noexcept
Test if the delegate is binded to the given object + non-static member function.

Static Public Member Functions

Create and bind (convenience methods).

Creates a delegate and binds it to the given target.

- template<TRetVal(*) (TParams...) TFunction>
static constexpr [Delegate](#) [CreateAndBind](#) () noexcept
Creates a delegate and binds it to the given (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
static constexpr [Delegate](#) [CreateAndBind](#) (TFunction &function) noexcept
Creates a delegate and binds it to the given functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
static constexpr [Delegate](#) [CreateAndBind](#) (TClass &object) noexcept
Creates a delegate and binds it to the given object + non-static member function.

6.4.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...)>
```

[Delegate](#) partial specialization for non-const.

Definition at line 216 of file [delegate.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.4.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate (
    const Delegate< TRetVal(TParams...)> & other) [constexpr], [noexcept]
```

6.4.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::~~Delegate () [default], [noexcept]
```

Remarks

Not virtual.

6.4.3 Member Function Documentation

6.4.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)>::operator= (
    const Delegate< TRetVal(TParams...)> & other) [constexpr], [noexcept]
```

6.4.3.2 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator== (
    const Delegate< TRetVal(TParams...)> & other) const [constexpr], [noexcept]
```

6.4.3.3 operator!=(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator!=(
    const Delegate< TRetVal(TParams...)> & other) const [constexpr], [noexcept]
```

6.4.3.4 Bind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
void bricxx::Delegate< TRetVal(TParams...)>::Bind () [constexpr], [noexcept]
```

Binds the delegate to a (static member) function.

6.4.3.5 Bind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
    TFunction & functor) [constexpr], [noexcept]
```

Binds the delegate to a functor / lambda (TFunction is auto-deduced).

6.4.3.6 Bind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
    TClass & object) [constexpr], [noexcept]
```

Binds the delegate to an object + non-static member function.

6.4.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo () const [constexpr], [noexcept]
```

Test if the delegate is binded to the given (static member) function.

6.4.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
    TFunction & functor) const [constexpr], [noexcept]
```

Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).

6.4.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
bool bricxx::Delegate<TRetVal(TParams...)>::IsBindedTo (
    TClass & object) const [constexpr], [noexcept]
```

Test if the delegate is binded to the given `object` + non-static member function.

6.4.3.10 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate<TRetVal(TParams...)>::operator() (
    UParams &&... params) [constexpr]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Exceptions

<i>BadDelegateCall</i>	if the delegate is not binded.
------------------------	--------------------------------

6.4.3.11 CreateAndBind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
constexpr Delegate bricxx::Delegate<TRetVal(TParams...)>::CreateAndBind () [static], [constexpr],
[noexcept]
```

Creates a delegate and binds it to the given (static member) function.

6.4.3.12 CreateAndBind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFuncor, typename = typename ::std::enable_if< ::std::is_class<TFuncor>::value >::type>
constexpr Delegate bricxx::Delegate<TRetVal(TParams...)>::CreateAndBind (
    TFuncor & functor) [static], [constexpr], [noexcept]
```

Creates a delegate and binds it to the given `functor` / `lambda` (`TFuncor` is auto-deduced).

6.4.3.13 CreateAndBind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
constexpr Delegate bricxx::Delegate< TRetVal(TParams...)>::CreateAndBind (
    TClass & object) [static], [constexpr], [noexcept]
```

Creates a delegate and binds it to the given `object` + non-static member function.

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference

Base class for [Delegate](#) partial specializations.

```
#include <delegate.hpp>
```

Public Member Functions

- constexpr bool [IsBinded](#) () const noexcept
- constexpr [operator bool](#) () const noexcept
Same as [IsBinded\(\)](#).
- constexpr void [Unbind](#) () noexcept
- constexpr void [Reset](#) () noexcept
Same as [Unbind\(\)](#).

Protected Types

- using [ErasedObjectType](#) = void
- using [StubFnPtr](#) = TRetVal (*)([ErasedObjectType](#)* pTypeErasedObject, TParams... params)

Protected Member Functions

- [~DelegateBase](#) () noexcept=default
- constexpr [DelegateBase](#) & [operator=](#) (const [DelegateBase](#) &other) noexcept

Constructors.

- constexpr [DelegateBase](#) () noexcept
Creates a delegate that is not binded.
- constexpr [DelegateBase](#) (const [DelegateBase](#) &other) noexcept

Equality operators.

- constexpr bool [operator==](#) (const [DelegateBase](#) &other) const noexcept
- constexpr bool [operator!=](#) (const [DelegateBase](#) &other) const noexcept

Static Protected Member Functions

- static TRetVal [NullStub](#) ([ErasedObjectType](#) *pTypeErasedObject, TParams... params)

Protected Attributes

- [StubFnPtr](#) pStub_ = &[NullStub](#)
- [ErasedObjectType](#) * pTypeErasedObject_ = nullptr

6.5.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::DelegateBase< TRetVal, TParams >
```

Base class for [Delegate](#) partial specializations.

Definition at line 172 of file [delegate.hpp](#).

6.5.2 Member Typedef Documentation

6.5.2.1 ErasedObjectType

```
template<typename TRetVal, typename... TParams>
using bricxx::DelegateBase< TRetVal, TParams >::ErasedObjectType = void [protected]
```

Definition at line 180 of file [delegate.hpp](#).

6.5.2.2 StubFnPtr

```
template<typename TRetVal, typename... TParams>
using bricxx::DelegateBase< TRetVal, TParams >::StubFnPtr = TRetVal (*) (ErasedObjectType* pTypeErasedObject, TParams... params) [protected]
```

Definition at line 181 of file [delegate.hpp](#).

6.5.3 Constructor & Destructor Documentation

6.5.3.1 DelegateBase() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase () [constexpr], [protected], [noexcept]
```

Creates a delegate that is not binded.

6.5.3.2 DelegateBase() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase (
    const DelegateBase< TRetVal, TParams > & other) [constexpr], [protected], [noexcept]
```

6.5.3.3 ~DelegateBase()

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::~~DelegateBase () [protected], [default], [noexcept]
```

Remarks

Not virtual.

6.5.4 Member Function Documentation

6.5.4.1 IsBinded()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::IsBinded () const [constexpr], [noexcept]
```

6.5.4.2 operator bool()

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::operator bool () const [explicit], [constexpr],
[noexcept]
```

Same as `IsBinded()`.

6.5.4.3 Unbind()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Unbind () [constexpr], [noexcept]
```

6.5.4.4 Reset()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Reset () [constexpr], [noexcept]
```

Same as `Unbind()`.

6.5.4.5 operator=()

```
template<typename TRetVal, typename... TParams>
DelegateBase & bricxx::DelegateBase< TRetVal, TParams >::operator= (
    const DelegateBase< TRetVal, TParams > & other) [constexpr], [protected], [noexcept]
```

6.5.4.6 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator==(
    const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.7 operator!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator!=(
    const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.8 NullStub()

```
template<typename TRetVal, typename... TParams>
TRetVal bricxx::DelegateBase< TRetVal, TParams >::NullStub (
    ErasedObjectType * pTypeErasedObject,
    TParams... params) [static], [protected]
```

Exceptions

<i>BadDelegateCall</i>	This stub is used when the delegate is not binded to a "real" target.
--	---

6.5.5 Member Data Documentation

6.5.5.1 pStub_

```
template<typename TRetVal, typename... TParams>
StubFnPtr bricxx::DelegateBase< TRetVal, TParams >::pStub_ = &NullStub [protected]
```

Definition at line 206 of file [delegate.hpp](#).

6.5.5.2 pTypeErasedObject_

```
template<typename TRetVal, typename... TParams>
ErasedObjectType* bricxx::DelegateBase< TRetVal, TParams >::pTypeErasedObject_ = nullptr [protected]
```

Definition at line 207 of file [delegate.hpp](#).

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

Chapter 7

File Documentation

7.1 `delegate.hpp` File Reference

```
#include <exception>
#include <type_traits>
#include <utility>
#include <memory>
```

Classes

- class `bricxx::BadDelegateCall`
Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class `bricxx::DelegateBase< TRetVal, TParams >`
Base class for `Delegate` partial specializations.
- class `bricxx::Delegate< TRetVal(TParams...) >`
`Delegate` partial specialization for non-const.
- class `bricxx::Delegate< TRetVal(TParams...) const >`
`Delegate` partial specialization for const.

Namespaces

- namespace `bricxx`

Macros

- `#define BRICXX_DELEGATE_API`

7.1.1 Macro Definition Documentation

7.1.1.1 `BRICXX_DELEGATE_API`

```
#define BRICXX_DELEGATE_API
```

Definition at line 58 of file `delegate.hpp`.

7.2 delegate.hpp

[Go to the documentation of this file.](#)

```

00001 // MIT License: https://spdx.org/licenses/MIT.html
00002 //
00003 // Copyright (c) 2026 Cristian ANITA <cristian_anita@yahoo.com>
00004 //
00005 // Permission is hereby granted, free of charge, to any person obtaining a copy
00006 // of this software and associated documentation files (the "Software"), to deal
00007 // in the Software without restriction, including without limitation the rights
00008 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00009 // copies of the Software, and to permit persons to whom the Software is
00010 // furnished to do so, subject to the following conditions:
00011 //
00012 // The above copyright notice and this permission notice shall be included in all
00013 // copies or substantial portions of the Software.
00014 //
00015 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
00016 // EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
00017 // MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
00018 // IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
00019 // DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
00020 // OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE
00021 // OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023
00024 #ifndef BRICXX_DELEGATE_HPP_INCLUDED_
00025 #define BRICXX_DELEGATE_HPP_INCLUDED_
00026
00027
00028 // /Zc:__cplusplus; please see:
00029 // - MSVC now correctly reports __cplusplus:
00030 // https://devblogs.microsoft.com/cppblog/msvc-now-correctly-reports-__cplusplus
00031 // - /Zc:__cplusplus (Enable updated __cplusplus macro):
00032 // https://learn.microsoft.com/en-us/cpp/build/reference/zc-cplusplus
00033 #ifdef _MSC_VER
00034 #   if (_MSC_VER < 1914)
00035 #       error "msvc >= 2017 v15.7 required!!! (for /Zc:__cplusplus)"
00036 #   endif
00037 #endif
00038
00039 #if (__cplusplus < 201402L)
00040 #   error "C++ >= 14 required!!!"
00041 #endif
00042
00043 #ifdef _MSC_VER
00044 #pragma warning(push)
00045 // C4646: function declared with __declspec(noreturn) has non-void return type.
00046 // DelegateBase::NullStub() has (possibly non-void) return type (it must conform to stub signature)
00047 // but it actually always throws BadDelegateCall.
00048 #pragma warning(disable: 4646)
00049 #endif
00050
00051 #include <exception>
00052 #include <type_traits>
00053 #include <utility>
00054 #include <memory>
00055
00056
00057 // TODO: If Delegate is used in a shared library define BRICXX_DELEGATE_API to that library
00058 // export/import macro.
00059 #define BRICXX_DELEGATE_API
00060
00061 namespace bricxx {
00062
00063
00064
00065 //*****
00066 template <typename TSignature>
00067 class Delegate;
00068
00069
00070
00071 //*****
00072 class BRICXX_DELEGATE_API BadDelegateCall final: public ::std::exception {
00073 public:
00074     const char* what() const noexcept override;
00075 };
00076
00077
00078

```

```

00167
00168
00169
00171 //*****
00172 template <typename TRetVal, typename... TParams>
00173 class DelegateBase {
00174 public:
00175     constexpr bool IsBinded() const noexcept;
00176     explicit constexpr operator bool() const noexcept;
00177     constexpr void Unbind() noexcept;
00178     constexpr void Reset() noexcept;
00179 protected:
00180     using ErasedObjectType = void;
00181     using StubFnPtr = TRetVal (*)(ErasedObjectType* pTypeErasedObject, TParams... params);
00182
00183     constexpr DelegateBase() noexcept;
00184     constexpr DelegateBase(const DelegateBase& other) noexcept;
00185
00186     ~DelegateBase() noexcept = default;
00187
00188     constexpr DelegateBase& operator=(const DelegateBase& other) noexcept;
00189
00190     constexpr bool operator==(const DelegateBase& other) const noexcept;
00191     constexpr bool operator!=(const DelegateBase& other) const noexcept;
00192
00193     [[noreturn]] static TRetVal NullStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00194
00195     StubFnPtr pStub_ = &NullStub;
00196     ErasedObjectType* pTypeErasedObject_ = nullptr;
00197 };
00198
00199
00200
00201
00202
00203 //*****
00204 template <typename TRetVal, typename... TParams>
00205 class Delegate<TRetVal (TParams...)> final: private DelegateBase<TRetVal, TParams...> {
00206 public:
00207     using DelegateBase<TRetVal, TParams...>::IsBinded;
00208     using DelegateBase<TRetVal, TParams...>::operator bool;
00209
00210     using DelegateBase<TRetVal, TParams...>::Unbind;
00211     using DelegateBase<TRetVal, TParams...>::Reset;
00212
00213     constexpr Delegate() noexcept;
00214     constexpr Delegate(const Delegate& other) noexcept;
00215
00216     ~Delegate() noexcept = default;
00217
00218     constexpr Delegate& operator=(const Delegate& other) noexcept;
00219
00220     constexpr bool operator==(const Delegate& other) const noexcept;
00221     constexpr bool operator!=(const Delegate& other) const noexcept;
00222
00223     template <TRetVal (*TFunction)(TParams...)>
00224     constexpr void Bind() noexcept;
00225
00226     template <class TFunction,
00227             typename = typename ::std::enable_if<
00228                 ::std::is_class<TFunction>::value
00229             >::type
00230     >
00231     constexpr void Bind(TFunction& functor) noexcept;
00232
00233     template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00234     constexpr void Bind(TClass& object) noexcept;
00235
00236
00237     template <TRetVal (*TFunction)(TParams...)>
00238     constexpr bool IsBindedTo() const noexcept;
00239
00240     template <class TFunction,
00241             typename = typename ::std::enable_if<
00242                 ::std::is_class<TFunction>::value
00243             >::type
00244     >

```

```

00276     >
00277     constexpr bool IsBindedTo(TFunction& functor) const noexcept;
00278
00280     template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00281     constexpr bool IsBindedTo(TClass& object) const noexcept;
00282
00284
00285
00290     template <typename... UParams>
00291     constexpr TRetVal operator()(UParams&&... params);
00292
00293
00297
00299     template <TRetVal (*TFunction)(TParams...)>
00300     static constexpr Delegate CreateAndBind() noexcept;
00301
00303     template <class TFunction,
00304               typename = typename ::std::enable_if<
00305                   ::std::is_class<TFunction>::value
00306                   >::type
00307               >
00308     static constexpr Delegate CreateAndBind(TFunction& functor) noexcept;
00309
00311     template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00312     static constexpr Delegate CreateAndBind(TClass& object) noexcept;
00313
00315 private:
00316     using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
00317     using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
00318
00319
00322
00323     template <TRetVal (*TFunction)(TParams...)>
00324     static constexpr TRetVal FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00325
00326     template <class TFunction>
00327     static constexpr TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00328
00329     template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00330     static constexpr TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00331
00333 };
00334
00335
00336
00337
00338
00340 //*****
00341 template <typename TRetVal, typename... TParams>
00342 class Delegate<TRetVal (TParams...) const> final: private DelegateBase<TRetVal, TParams...> {
00343 public:
00344     using DelegateBase<TRetVal, TParams...>::IsBinded;
00345     using DelegateBase<TRetVal, TParams...>::operator bool;
00346
00347     using DelegateBase<TRetVal, TParams...>::Unbind;
00348     using DelegateBase<TRetVal, TParams...>::Reset;
00349
00352     constexpr Delegate() noexcept;
00353     constexpr Delegate(const Delegate& other) noexcept;
00355
00356     ~Delegate() noexcept = default;
00357
00358     constexpr Delegate& operator=(const Delegate& other) noexcept;
00359
00360
00363     constexpr bool operator==(const Delegate& other) const noexcept;
00364     constexpr bool operator!=(const Delegate& other) const noexcept;
00366
00367
00371
00372     template <TRetVal (*TFunction)(TParams...)>
00373     constexpr void Bind() noexcept;
00374
00376     template <class TFunction,
00377               typename = typename ::std::enable_if<
00378                   ::std::is_class<TFunction>::value
00379                   >::type
00380               >
00381     constexpr void Bind(const TFunction& functor) noexcept;
00382
00383     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00384     constexpr void Bind(const TClass& object) noexcept;
00385
00387
00388
00392

```

```

00393     template <TRetVal (*TFunction)(TParams...)>
00394     constexpr bool IsBindedTo() const noexcept;
00395
00397     template <class TFunction,
00398             typename = typename ::std::enable_if<
00399                 ::std::is_class<TFunction>::value
00400             >::type
00401     >
00402     constexpr bool IsBindedTo(const TFunction& functor) const noexcept;
00403
00405     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00406     constexpr bool IsBindedTo(const TClass& object) const noexcept;
00407
00409
00410
00415     template <typename... UParams>
00416     constexpr TRetVal operator()(UParams&&... params) const;
00417
00418
00422
00424     template <TRetVal (*TFunction)(TParams...)>
00425     static constexpr Delegate CreateAndBind() noexcept;
00426
00428     template <class TFunction,
00429             typename = typename ::std::enable_if<
00430                 ::std::is_class<TFunction>::value
00431             >::type
00432     >
00433     static constexpr Delegate CreateAndBind(const TFunction& functor) noexcept;
00434
00436     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00437     static constexpr Delegate CreateAndBind(const TClass& object) noexcept;
00438
00440 private:
00441     using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
00442     using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
00443
00444
00447
00448     template <TRetVal (*TFunction)(TParams...)>
00449     static constexpr TRetVal FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00450
00451     template <class TFunction>
00452     static constexpr TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00453
00454     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00455     static constexpr TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00456
00458 };
00459
00460
00461
00462
00464
00465
00466
00467 // *****
00468 // Inline functions / template implementation
00469 inline const char* BadDelegateCall::what() const noexcept {
00470     return "Bad delegate call";
00471 }
00472
00473
00474
00475
00476 template <typename TRetVal, typename... TParams>
00477 inline constexpr bool DelegateBase<TRetVal, TParams...>::IsBinded() const noexcept {
00478     return pStub_ != &NullStub;
00479 }
00480
00481 template <typename TRetVal, typename... TParams>
00482 inline constexpr DelegateBase<TRetVal, TParams...>::operator bool() const noexcept {
00483     return IsBinded();
00484 }
00485
00486
00487 template <typename TRetVal, typename... TParams>
00488 inline constexpr void DelegateBase<TRetVal, TParams...>::Unbind() noexcept {
00489     pStub_ = &NullStub;
00490     pTypeErasedObject_ = nullptr;
00491 }
00492
00493 template <typename TRetVal, typename... TParams>
00494 inline constexpr void DelegateBase<TRetVal, TParams...>::Reset() noexcept {
00495     Unbind();
00496 }

```

```

00497
00498
00499 template <typename TRetVal, typename... TParams>
00500 inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase() noexcept = default;
00501
00502 template <typename TRetVal, typename... TParams>
00503 inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase(const DelegateBase& other) noexcept =
    default;
00504
00505
00506 template <typename TRetVal, typename... TParams>
00507 inline constexpr DelegateBase<TRetVal, TParams...>& DelegateBase<TRetVal, TParams...>::operator=(
00508     const DelegateBase& other) noexcept = default;
00509
00510
00511 template <typename TRetVal, typename... TParams>
00512 inline constexpr bool DelegateBase<TRetVal, TParams...>::operator==(const DelegateBase& other) const
    noexcept {
00513     return (pStub_ == other.pStub_) && (pTypeErasedObject_ == other.pTypeErasedObject_);
00514 }
00515
00516 template <typename TRetVal, typename... TParams>
00517 inline constexpr bool DelegateBase<TRetVal, TParams...>::operator!=(const DelegateBase& other) const
    noexcept {
00518     return !(*this == other);
00519 }
00520
00521
00522 template <typename TRetVal, typename... TParams>
00523 TRetVal DelegateBase<TRetVal, TParams...>::NullStub(ErasedObjectType* pTypeErasedObject, TParams...
    params) {
00524     (void)pTypeErasedObject;
00525     (void)sizeof...(params);
00526
00527     throw BadDelegateCall();
00528 }
00529
00530
00531
00532
00533 template <typename TRetVal, typename... TParams>
00534 inline constexpr Delegate<TRetVal (TParams...)>::Delegate() noexcept = default;
00535
00536 template <typename TRetVal, typename... TParams>
00537 inline constexpr Delegate<TRetVal (TParams...)>::Delegate(const Delegate& other) noexcept = default;
00538
00539
00540 template <typename TRetVal, typename... TParams>
00541 inline constexpr Delegate<TRetVal (TParams...)>& Delegate<TRetVal (TParams...)>::operator=(const
    Delegate& other)
00542     noexcept = default;
00543
00544
00545 template <typename TRetVal, typename... TParams>
00546 inline constexpr bool Delegate<TRetVal (TParams...)>::operator==(const Delegate& other) const noexcept
    {
00547     return DelegateBase<TRetVal, TParams...>::operator==(other);
00548 }
00549
00550 template <typename TRetVal, typename... TParams>
00551 inline constexpr bool Delegate<TRetVal (TParams...)>::operator!=(const Delegate& other) const noexcept
    {
00552     return !(*this == other);
00553 }
00554
00555
00556 template <typename TRetVal, typename... TParams>
00557 template <TRetVal (*TFunction)(TParams...)>
00558 inline constexpr void Delegate<TRetVal (TParams...)>::Bind() noexcept {
00559     this->pStub_ = &FunctionStub<TFunction>;
00560     this->pTypeErasedObject_ = nullptr;
00561 }
00562
00563 template <typename TRetVal, typename... TParams>
00564 template <class TFunction, typename>
00565 inline constexpr void Delegate<TRetVal (TParams...)>::Bind(TFunction& functor) noexcept {
00566     this->pStub_ = &FunctionStub<TFunction>;
00567     this->pTypeErasedObject_ = ::std::addressof(functor);
00568 }
00569
00570 template <typename TRetVal, typename... TParams>
00571 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00572 inline constexpr void Delegate<TRetVal (TParams...)>::Bind(TClass& object) noexcept {
00573     static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");
00574
00575     this->pStub_ = &MethodStub<TClass, TMethod>;
00576     this->pTypeErasedObject_ = ::std::addressof(object);

```

```

00577 }
00578
00579
00580 template <typename TRetVal, typename... TParams>
00581 template <TRetVal (*TFunction)(TParams...)>
00582 inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo() const noexcept {
00583     Delegate delegate;
00584     delegate.Bind<TFunction>();
00585     return *this == delegate;
00586 }
00587
00588 template <typename TRetVal, typename... TParams>
00589 template <class TFunction, typename>
00590 inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo(TFunction& functor) const noexcept {
00591     Delegate delegate;
00592     delegate.Bind(functor);
00593     return *this == delegate;
00594 }
00595
00596 template <typename TRetVal, typename... TParams>
00597 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00598 inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo(TClass& object) const noexcept {
00599     Delegate delegate;
00600     delegate.Bind<TClass, TMethod>(object);
00601     return *this == delegate;
00602 }
00603
00604
00605 template <typename TRetVal, typename... TParams>
00606 template <typename... UParams>
00607 inline constexpr TRetVal Delegate<TRetVal (TParams...)>::operator()(UParams&&... params) {
00608     return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
00609 }
00610
00611
00612 template <typename TRetVal, typename... TParams>
00613 template <TRetVal (*TFunction)(TParams...)>
00614 inline constexpr Delegate<TRetVal (TParams...)> Delegate<TRetVal (TParams...)>::CreateAndBind()
00615     noexcept {
00616     Delegate delegate;
00617     delegate.Bind<TFunction>();
00618     return delegate;
00619 }
00620
00621 template <typename TRetVal, typename... TParams>
00622 template <class TFunction, typename>
00623 inline constexpr Delegate<TRetVal (TParams...)> Delegate<TRetVal
00624     (TParams...)>::CreateAndBind(TFunction& functor) noexcept {
00625     Delegate delegate;
00626     delegate.Bind(functor);
00627     return delegate;
00628 }
00629
00630 template <typename TRetVal, typename... TParams>
00631 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00632 inline constexpr Delegate<TRetVal (TParams...)> Delegate<TRetVal (TParams...)>::CreateAndBind(TClass&
00633     object) noexcept {
00634     Delegate delegate;
00635     delegate.Bind<TClass, TMethod>(object);
00636     return delegate;
00637 }
00638
00639 template <typename TRetVal, typename... TParams>
00640 template <TRetVal (*TFunction)(TParams...)>
00641 constexpr TRetVal Delegate<TRetVal (TParams...)>::FunctionStub(ErasedObjectType* pTypeErasedObject,
00642     TParams... params) {
00643     (void)pTypeErasedObject;
00644     return (*TFunction)(::std::forward<TParams>(params)...);
00645 }
00646
00647 template <typename TRetVal, typename... TParams>
00648 template <class TFunction>
00649 constexpr TRetVal Delegate<TRetVal (TParams...)>::FunctorStub(ErasedObjectType* pTypeErasedObject,
00650     TParams... params) {
00651     return (*static_cast<TFunction*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
00652 }
00653
00654 template <typename TRetVal, typename... TParams>
00655 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00656 constexpr TRetVal Delegate<TRetVal (TParams...)>::MethodStub(ErasedObjectType* pTypeErasedObject,
00657     TParams... params) {
00658     return (*static_cast<TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
00659 }
00660
00661
00662

```

```

00658
00659
00660 template <typename TRetVal, typename... TParams>
00661 inline constexpr Delegate<TRetVal (TParams...) const>::Delegate() noexcept = default;
00662
00663 template <typename TRetVal, typename... TParams>
00664 inline constexpr Delegate<TRetVal (TParams...) const>::Delegate(const Delegate& other) noexcept =
    default;
00665
00666
00667 template <typename TRetVal, typename... TParams>
00668 inline constexpr Delegate<TRetVal (TParams...) const>& Delegate<TRetVal (TParams...)
    const>::operator=(const Delegate& other)
00669     noexcept = default;
00670
00671
00672 template <typename TRetVal, typename... TParams>
00673 inline constexpr bool Delegate<TRetVal (TParams...) const>::operator==(const Delegate& other) const
    noexcept {
00674     return DelegateBase<TRetVal, TParams...>::operator==(other);
00675 }
00676
00677 template <typename TRetVal, typename... TParams>
00678 inline constexpr bool Delegate<TRetVal (TParams...) const>::operator!=(const Delegate& other) const
    noexcept {
00679     return !(*this == other);
00680 }
00681
00682
00683 template <typename TRetVal, typename... TParams>
00684 template <TRetVal (*TFunction) (TParams...)>
00685 inline constexpr void Delegate<TRetVal (TParams...) const>::Bind() noexcept {
00686     this->pStub_ = &FunctionStub<TFunction>;
00687     this->pTypeErasedObject_ = nullptr;
00688 }
00689
00690 template <typename TRetVal, typename... TParams>
00691 template <class TFuncor, typename>
00692 inline constexpr void Delegate<TRetVal (TParams...) const>::Bind(const TFuncor& funcor) noexcept {
00693     this->pStub_ = &FuncorStub<TFuncor>;
00694
00695     // removed const will be restored in FuncorStub()
00696     this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const
    ErasedObjectType*>(&std::addressof(funcor)));
00697 }
00698
00699 template <typename TRetVal, typename... TParams>
00700 template <class TClass, TRetVal (TClass::*TMethod) (TParams...) const>
00701 inline constexpr void Delegate<TRetVal (TParams...) const>::Bind(const TClass& object) noexcept {
00702     static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");
00703
00704     this->pStub_ = &MethodStub<TClass, TMethod>;
00705
00706     // removed const will be restored in MethodStub()
00707     this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const
    ErasedObjectType*>(&std::addressof(object)));
00708 }
00709
00710
00711 template <typename TRetVal, typename... TParams>
00712 template <TRetVal (*TFunction) (TParams...)>
00713 inline constexpr bool Delegate<TRetVal (TParams...) const>::IsBindedTo() const noexcept {
00714     Delegate delegate;
00715     delegate.Bind<TFunction>();
00716     return *this == delegate;
00717 }
00718
00719 template <typename TRetVal, typename... TParams>
00720 template <class TFuncor, typename>
00721 inline constexpr bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TFuncor& funcor) const
    noexcept {
00722     Delegate delegate;
00723     delegate.Bind(funcor);
00724     return *this == delegate;
00725 }
00726
00727 template <typename TRetVal, typename... TParams>
00728 template <class TClass, TRetVal (TClass::*TMethod) (TParams...) const>
00729 inline constexpr bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TClass& object) const
    noexcept {
00730     Delegate delegate;
00731     delegate.Bind<TClass, TMethod>(object);
00732     return *this == delegate;
00733 }
00734
00735
00736 template <typename TRetVal, typename... TParams>

```



```

00737 template <typename... UParams>
00738 inline constexpr TRetVal Delegate<TRetVal (TParams...) const>::operator()(UParams&&... params) const {
00739     return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
00740 }
00741
00742
00743 template <typename TRetVal, typename... TParams>
00744 template <TRetVal (*TFunction)(TParams...)>
00745 inline constexpr Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...)
const>::CreateAndBind() noexcept {
00746     Delegate delegate;
00747     delegate.Bind<TFunction>();
00748     return delegate;
00749 }
00750
00751 template <typename TRetVal, typename... TParams>
00752 template <class TFuncutor, typename>
00753 inline constexpr Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...)
const>::CreateAndBind(
00754     const TFuncutor& functor) noexcept {
00755     Delegate delegate;
00756     delegate.Bind(functor);
00757     return delegate;
00758 }
00759
00760 template <typename TRetVal, typename... TParams>
00761 template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00762 inline constexpr Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...)
const>::CreateAndBind(
00763     const TClass& object) noexcept {
00764     Delegate delegate;
00765     delegate.Bind<TClass, TMethod>(object);
00766     return delegate;
00767 }
00768
00769
00770 template <typename TRetVal, typename... TParams>
00771 template <TRetVal (*TFunction)(TParams...)>
00772 constexpr TRetVal Delegate<TRetVal (TParams...) const>::FunctionStub(ErasedObjectType*
pTypeErasedObject, TParams... params) {
00773     (void)pTypeErasedObject;
00774
00775     return (*TFunction)(::std::forward<TParams>(params)...);
00776 }
00777
00778 template <typename TRetVal, typename... TParams>
00779 template <class TFuncutor>
00780 constexpr TRetVal Delegate<TRetVal (TParams...) const>::FuncutorStub(ErasedObjectType*
pTypeErasedObject, TParams... params) {
00781     // restoring removed const
00782     return (*static_cast<const TFuncutor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
00783 }
00784
00785 template <typename TRetVal, typename... TParams>
00786 template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00787 constexpr TRetVal Delegate<TRetVal (TParams...) const>::MethodStub(ErasedObjectType*
pTypeErasedObject, TParams... params) {
00788     // restoring removed const
00789     return (static_cast<const
TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
00790 }
00791
00792
00793
00794
00795
00796 } // namespace bricxx
00797
00798
00799 #ifdef _MSC_VER
00800 #pragma warning(pop)
00801 #endif
00802
00803
00804 #endif // BRICXX_DELEGATE_HPP_INCLUDED_

```


Index

~Delegate
 bricxx::Delegate< TRetVal(TParams...) const >, [15](#)
 bricxx::Delegate< TRetVal(TParams...) >, [20](#)
~DelegateBase
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

Bind
 bricxx::Delegate< TRetVal(TParams...) const >, [16](#)
 bricxx::Delegate< TRetVal(TParams...) >, [21](#)

bricxx, [9](#)

bricxx::BadDelegateCall, [11](#)
 what, [11](#)

bricxx::Delegate< TRetVal(TParams...) const >, [13](#)
 ~Delegate, [15](#)
 Bind, [16](#)
 CreateAndBind, [17](#), [18](#)
 Delegate, [15](#)
 IsBindedTo, [16](#), [17](#)
 operator!=, [16](#)
 operator(), [17](#)
 operator=, [16](#)
 operator==, [16](#)

bricxx::Delegate< TRetVal(TParams...) >, [18](#)
 ~Delegate, [20](#)
 Bind, [21](#)
 CreateAndBind, [22](#)
 Delegate, [20](#)
 IsBindedTo, [21](#)
 operator!=, [20](#)
 operator(), [22](#)
 operator=, [20](#)
 operator==, [20](#)

bricxx::Delegate< TSignature >, [12](#)

bricxx::DelegateBase< TRetVal, TParams >, [23](#)
 ~DelegateBase, [25](#)
 DelegateBase, [24](#)
 ErasedObjectType, [24](#)
 IsBinded, [25](#)
 NullStub, [26](#)
 operator bool, [25](#)
 operator!=, [26](#)
 operator=, [25](#)
 operator==, [25](#)
 pStub_, [26](#)
 pTypeErasedObject_, [26](#)
 Reset, [25](#)
 StubFnPtr, [24](#)
 Unbind, [25](#)

BRICXX_DELEGATE_API
 delegate.hpp, [27](#)

CreateAndBind
 bricxx::Delegate< TRetVal(TParams...) const >, [17](#), [18](#)
 bricxx::Delegate< TRetVal(TParams...) >, [22](#)

Delegate
 bricxx::Delegate< TRetVal(TParams...) const >, [15](#)
 bricxx::Delegate< TRetVal(TParams...) >, [20](#)
 delegate.hpp, [27](#)
 BRICXX_DELEGATE_API, [27](#)

DelegateBase
 bricxx::DelegateBase< TRetVal, TParams >, [24](#)

ErasedObjectType
 bricxx::DelegateBase< TRetVal, TParams >, [24](#)

IsBinded
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

IsBindedTo
 bricxx::Delegate< TRetVal(TParams...) const >, [16](#), [17](#)
 bricxx::Delegate< TRetVal(TParams...) >, [21](#)

NullStub
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)

operator bool
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

operator!=
 bricxx::Delegate< TRetVal(TParams...) const >, [16](#)
 bricxx::Delegate< TRetVal(TParams...) >, [20](#)
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)

operator()
 bricxx::Delegate< TRetVal(TParams...) const >, [17](#)
 bricxx::Delegate< TRetVal(TParams...) >, [22](#)

operator=
 bricxx::Delegate< TRetVal(TParams...) const >, [16](#)
 bricxx::Delegate< TRetVal(TParams...) >, [20](#)
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

operator==
 bricxx::Delegate< TRetVal(TParams...) const >, [16](#)
 bricxx::Delegate< TRetVal(TParams...) >, [20](#)
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

pStub_
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)
pTypeErasedObject_
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)

Reset
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

StubFnPtr
 bricxx::DelegateBase< TRetVal, TParams >, [24](#)

Unbind
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

what
 bricxx::BadDelegateCall, [11](#)